Chocolate Distribution Problem

Difficulty: **Easy** Accuracy: **49.91%** Submissions: **279K+** Points: **2** Average Time: **15m**

Given an array **arr[]** of positive integers, where each value represents the number of chocolates in a packet. Each packet can have a variable number of chocolates. There are **m** students, the task is to distribute chocolate packets among **m** students such that -

i. Each student gets **exactly** one packet.

ii. The difference between maximum number of chocolates given to a student and minimum number of chocolates given to a student is minimum and return that minimum possible difference.

Examples:

Input: arr = [3, 4, 1, 9, 56, 7, 9, 12], m = 5

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed

1112 / 1112

Attempts : Correct / Total

1 / 1

Accuracy : 100%

Points Scored

Time Taken

Java (21) Start Timer

```java
import java.util.*;

class Solution {
    public long findMinDiff (ArrayList<Integer> a, int m) {

        Collections.sort(a);

        int n = a.size();
        long minDiff = Long.MAX_VALUE;

        for (int i = 0; i + m - 1 < n; i++) {

            long diff = a.get(i + m - 1) - a.get(i);
            minDiff = Math.min(minDiff, diff);
        }

        return minDiff;
    }
}
```

Custom Input Compile & Run Submit

Smallest subarray with sum greater than x

Difficulty: **Easy** Accuracy: **37.07%** Submissions: **155K+** Points: **2** Average Time: **20m**

Given a number **x** and an array of integers **arr**, find the smallest subarray with sum greater than the given value. If such a subarray do not exist return 0 in that case.

Examples:

Input: x = 51, arr[] = [1, 4, 45, 6, 0, 19]
Output: 3
Explanation: Minimum length subarray is [4, 45, 6]

Input: x = 100, arr[] = [1, 10, 5, 2, 7]
Output: 0

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1112 / 1112	**1 / 1**
	Accuracy : 100%

Points Scored Time Taken

Java (21) Start Timer

```java
class Solution {

    public static int smallestSubWithSum(int x, int[] arr) {

        int n = arr.length;
        int start = 0;
        int sum = 0;
        int minLength = Integer.MAX_VALUE;

        for (int end = 0; end < n; end++) {

            sum += arr[end];

            while (sum > x) {
                minLength = Math.min(minLength, end - start + 1);
                sum -= arr[start];
                start++;
            }
        }

        if (minLength == Integer.MAX_VALUE)
            return 0;

        return minLength;
    }
}
```

Custom Input Compile & Run Submit

Three way partitioning

Difficulty: **Easy** Accuracy: **41.58%** Submissions: **188K+** Points: **2** Average Time: **20m**

Given an **array** and a range **a**, **b**. The task is to partition the array around the range such that the array is divided into three parts.
1) All elements smaller than **a** come first.
2) All elements in range **a** to **b** come next.
3) All elements greater than **b** appear in the end.
The individual elements of three sets can appear in any order. You are required to return the modified array.

Note: The generated output is true if you modify the given array successfully. Otherwise false.

Geeky Challenge: Solve this problem in O(n) time complexity.

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1111 / 1111	**1 / 1** Accuracy : 100%

Points Scored Time Taken

Java (21) Start Timer

```java
class Solution {
    public void threeWayPartition(int[] arr, int a, int b) {

        int low = 0;
        int mid = 0;
        int high = arr.length - 1;

        while (mid <= high) {

            if (arr[mid] < a) {
                int temp = arr[low];
                arr[low] = arr[mid];
                arr[mid] = temp;
                low++;
                mid++;
            }
            else if (arr[mid] > b) {
                int temp = arr[mid];
                arr[mid] = arr[high];
                arr[high] = temp;
                high--;
            }
            else {
                mid++;
            }
        }
    }
}
```

Custom Input Compile & Run Submit

Minimum swaps and K together

Difficulty: **Medium** Accuracy: **26.0%** Submissions: **142K+** Points: **4**

Given an array **arr** and a number **k**. One can apply a swap operation on the array any number of times, i.e choose any two index i and j (i < j) and swap arr[i] , arr[j] . Find the **minimum** number of swaps required to bring all the numbers less than or equal to **k** together, i.e. make them a contiguous subarray.

Examples :

Input: arr[] = [2, 1, 5, 6, 3], k = 3
Output: 1
Explanation: To bring elements 2, 1, 3 together, swap index 2 with 4 (0-based indexing), i.e. element arr[2] = 5 with arr[4] = 3 such that final array will be- arr[] =

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1112 / 1112

Attempts : Correct / Total
1 / 1
Accuracy : 100%

Points Scored

Time Taken

Java (21) Start Timer

```java
class Solution {
    public int minSwap(int arr[], int k) {

        int n = arr.length;

        int good = 0;
        for (int i = 0; i < n; i++) {
            if (arr[i] <= k)
                good++;
        }

        int bad = 0;
        for (int i = 0; i < good; i++) {
            if (arr[i] > k)
                bad++;
        }

        int ans = bad;

        int i = 0, j = good;
        while (j < n) {

            if (arr[i] > k)
                bad--;
            if (arr[j] > k)
                bad++;

            ans = Math.min(ans, bad);

            i++;
            j++;
        }

        return ans;
    }
}
```

Custom Input Compile & Run Submit

Median of an Array

Difficulty: **Basic** Accuracy: **44.57%** Submissions: **153K+** Points: **1**

Given an array **arr[]** of integers, calculate the median.

Examples:

Input: arr[] = [90, 100, 78, 89, 67]
Output: 89
Explanation: After sorting the array middle element is the median

Input: arr[] = [56, 67, 30, 79]
Output: 61.5
Explanation: In case of even number of elements, average of two middle elements is

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1115 / 1115	**1 / 1**
	Accuracy : 100%

Points Scored Time Taken

Java (21) Start Timer

```java
import java.util.*;

class Solution {
    public double findMedian(int[] arr) {
        Arrays.sort(arr);
        int n = arr.length;

        if (n % 2 == 1) {
            return arr[n/2];
        } else {
            return (arr[n/2] + arr[n/2 - 1]) / 2.0;
        }
    }
}
```

Custom Input Compile & Run Submit

Spirally traversing a matrix

Difficulty: **Medium** Accuracy: **35.2%** Submissions: **343K+** Points: **4**

You are given a rectangular matrix **mat[][]** of size **n x m**, and your task is to return an array while **traversing** the matrix in **spiral** form.

Examples:

Input: mat[][] = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 15, 16]]
Output: [1, 2, 3, 4, 8, 12, 16, 15, 14, 13, 9, 5, 6, 7, 11, 10]
Explanation:

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1115 / 1115	**1 / 1**
	Accuracy : 100%

Points Scored Time Taken

Java (21) Start Timer

```java
class Solution {
    public ArrayList<Integer> spirallyTraverse(int matrix[][]) {
        ArrayList<Integer> res = new ArrayList<>();

        int r = matrix.length;
        int c = matrix[0].length;

        int top = 0, bottom = r - 1;
        int left = 0, right = c - 1;

        while (top <= bottom && left <= right) {

            for (int i = left; i <= right; i++)
                res.add(matrix[top][i]);
            top++;

            for (int i = top; i <= bottom; i++)
                res.add(matrix[i][right]);
            right--;

            if (top <= bottom) {
                for (int i = right; i >= left; i--)
                    res.add(matrix[bottom][i]);
                bottom--;
            }

            if (left <= right) {
                for (int i = bottom; i >= top; i--)
                    res.add(matrix[i][left]);
                left++;
            }
        }

        return res;
    }
}
```

Custom Input Compile & Run Submit

74. Search a 2D Matrix

Medium | Topics | Companies

You are given an `m x n` integer matrix `matrix` with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer `target`, return `true` *if* `target` *is in* `matrix` *or* `false` *otherwise.*

You must write a solution in `O(log(m * n))` time complexity.

Example 1:

1	3	5	7
10	11	16	20
23	30	34	60

```
Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
Output: true
```

Example 2:

17.8K | 350 | 87 Online

Code

Java | Auto

```java
        while (i < matrix.length && j >= 0) {
            if (matrix[i][j] == target)
                return true;
            else if (target > matrix[i][j])
                i++;
            else
                j--;
        }

        return false;
    }
}
```

Saved | Ln 17, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

matrix =
```
[[1,3,5,7],[10,11,16,20],[23,30,34,60]]
```

target =
```
3
```

Output

```
true
```

Median in a row-wise sorted Matrix

Difficulty: **Medium** Accuracy: **55.05%** Submissions: **172K+** Points: **4**

Given a **row-wise sorted** matrix **mat[][]** of size n*m, where the number of rows and columns is always **odd**. Return the **median** of the matrix.

Examples:

Input: mat[][] = [[1, 3, 5],
[2, 6, 9],
[3, 6, 9]]
Output: 5
Explanation: Sorting matrix elements gives us [1, 2, 3, 3, 5, 6, 6, 9, 9]. Hence, 5 is median.

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1117 / 1117	**1 / 1** Accuracy : 100%

Points Scored Time Taken

Java (21) Start Timer

```java
        int high = Integer.MIN_VALUE;

        for (int i = 0; i < R; i++) {
            low = Math.min(low, mat[i][0]);
            high = Math.max(high, mat[i][C - 1]);
        }

        int desired = (R * C) / 2;

        while (low <= high) {
            int mid = low + (high - low) / 2;
            int count = 0;

            for (int i = 0; i < R; i++) {
                count += countSmallerEqual(mat[i], mid);
            }

            if (count <= desired)
                low = mid + 1;
            else
                high = mid - 1;
        }

        return low;
    }

    private int countSmallerEqual(int[] row, int target) {
        int low = 0, high = row.length - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            if (row[mid] <= target)
                low = mid + 1;
            else
                high = mid - 1;
        }

        return low;
    }
}
```

Custom Input Compile & Run Submit

Row with max 1s

Difficulty: **Medium** Accuracy: **33.09%** Submissions: **378K+** Points: **4**

You are given a 2D binary array **arr[][]** consisting of only 1s and 0s. Each row of the array is sorted in non-decreasing order. Your task is to find and return the index of the first row that contains the maximum number of 1s. If no such row exists, return -1.

Note:

- The array follows 0-based indexing.
- The number of rows and columns in the array are denoted by n and m respectively.

Examples:

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1111 / 1111	**1 / 1**
	Accuracy : 100%

Points Scored Time Taken

Java (21) Start Timer

```java
class Solution {
    int rowWithMax1s(int arr[][]) {

        int n = arr.length;
        int m = arr[0].length;

        int maxRow = -1;
        int j = m - 1;

        for (int i = 0; i < n; i++) {
            while (j >= 0 && arr[i][j] == 1) {
                j--;
                maxRow = i;
            }
        }

        return maxRow;
    }
}
```

Custom Input Compile & Run Submit